

✓ 기본 라이브러리 (패키지, 모듈) 불러오기

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
os.getcwd()
```

```
↳ '/content'
```

#Colab에서 Mecab-ko-dic를 쉽게 설치할 수 있도록 도와주는 Shell-Script.
!git clone <https://github.com/SOMJANG/Mecab-ko-for-Google-Colab.git>

```
↳ Cloning into 'Mecab-ko-for-Google-Colab'...
remote: Enumerating objects: 138, done.
remote: Counting objects: 100% (47/47), done.
remote: Compressing objects: 100% (38/38), done.
remote: Total 138 (delta 26), reused 22 (delta 8), pack-reused 91 (from 1)
Receiving objects: 100% (138/138), 1.72 MiB | 18.11 MiB/s, done.
Resolving deltas: 100% (65/65), done.
```

```
cd Mecab-ko-for-Google-Colab
```

```
↳ /content/Mecab-ko-for-Google-Colab
```

```
▶!bash install_mecab-ko_on_colab_light_220429.sh
```

```
... Installing konlpy.....
Collecting konlpy
  Downloading konlpy-0.6.0-py2.py3-none-any.whl.metadata (1.9 kB)
Collecting JPype1>=0.7.0 (from konlpy)
  Downloading JPype1-1.5.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (4.9 kB)
Requirement already satisfied: lxml>=4.1.0 in /usr/local/lib/python3.10/dist-packages (from konlpy) (4.9.4)
Requirement already satisfied: numpy>=1.6 in /usr/local/lib/python3.10/dist-packages (from konlpy) (1.26.4)
Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (from JPype1>=0.7.0->konlpy) (24.1)
Downloading konlpy-0.6.0-py2.py3-none-any.whl (19.4 MB)
----- 19.4/19.4 MB 65.2 MB/s eta 0:00:00
Downloading JPype1-1.5.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (488 kB)
----- 488.6/488.6 kB 26.0 MB/s eta 0:00:00

Installing collected packages: JPype1, konlpy
Successfully installed JPype1-1.5.0 konlpy-0.6.0
Done
Installing mecab-0.996-ko-0.9.2.tar.gz.....
Downloading mecab-0.996-ko-0.9.2.tar.gz.....
from https://bitbucket.org/eunjeon/mecab-ko/downloads/mecab-0.996-ko-0.9.2.tar.gz
--2024-10-02 10:48:43-- https://bitbucket.org/eunjeon/mecab-ko/downloads/mecab-0.996-ko-0.9.2.tar.gz
Resolving bitbucket.org (bitbucket.org)... 104.192.142.26, 104.192.142.25, 104.192.142.24, ...
Connecting to bitbucket.org (bitbucket.org)|104.192.142.26|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://bbuseruploads.s3.amazonaws.com/eunjeon/mecab-ko/downloads/mecab-0.996-ko-0.9.2.tar.gz?response-content-disposit
--2024-10-02 10:48:43-- https://bbuseruploads.s3.amazonaws.com/eunjeon/mecab-ko/downloads/mecab-0.996-ko-0.9.2.tar.gz?response-c
Resolving bbuseruploads.s3.amazonaws.com (bbuseruploads.s3.amazonaws.com)... 52.216.41.97, 3.5.28.155, 16.182.39.153, ...
Connecting to bbuseruploads.s3.amazonaws.com (bbuseruploads.s3.amazonaws.com)|52.216.41.97|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1414979 (1.3M) [application/x-tar]
Saving to: 'mecab-0.996-ko-0.9.2.tar.gz'

mecab-0.996-ko-0.9. 100%[=====>] 1.35M --.-KB/s in 0.1s

2024-10-02 10:48:43 (12.0 MB/s) - 'mecab-0.996-ko-0.9.2.tar.gz' saved [1414979/1414979]

Done
Unpacking mecab-0.996-ko-0.9.2.tar.gz.....
Done
Change Directory to mecab-0.996-ko-0.9.2.....
installing mecab-0.996-ko-0.9.2.tar.gz.....
configure
make
make check
```

```
make install
```

```
#Colab에서 konlpy 설치
#!curl -s https://raw.githubusercontent.com/teddylee777/machine-learning/master/99-Misc/01-Colab/mecab-colab.sh | bash
import konlpy
# Tokenization and POS for korean
from konlpy.tag import Kkma
from konlpy.tag import Okt
from konlpy.tag import Hannanum
from konlpy.tag import Komoran
from konlpy.tag import Mecab
```

```
kkma = Kkma()
okt = Okt()
Hnn = Hannanum()
mecab=Mecab()
```

```
Image('./형태소 분석 라이브러리.png')
```

☞ 형태소 분석

KoNLPy는 다음과 같은 다양한 형태소 분석, 태깅 라이브러리를 파이썬에서 쉽게 사용할 수 있도록 모아놓았다.

- Hannanum: 한나눔. KAIST Semantic Web Research Center 개발.
 - <http://semanticweb.kaist.ac.kr/hannanum/>
- Kkma: 꼬꼬마. 서울대학교 IDS(Intelligent Data Systems) 연구실 개발.
 - <http://kkma.snu.ac.kr/>
- Komoran: 코모란. Shineware에서 개발.
 - <https://github.com/shin285/KOMORAN>
- Mecab: 메카브. 일본어용 형태소 분석기를 한국어를 사용할 수 있도록 수정.
 - <https://bitbucket.org/eunjeon/mecab-ko>
- Open Korean Text: 오픈 소스 한국어 분석기. 과거 트위터 형태소 분석기.
 - <https://github.com/open-korean-text/open-korean-text>

✓ 한국어 문장토크

```
text1 = '학생이 자연어처리 수업을 듣는다'
```

```
text2 = '학생이 자연어처리 수업을 듣다'
```

```
mecab.morphs(text1)
```

```
☞ ['학생', '이', '자연어', '처리', '수업', '을', '듣', '는다']
```

```
mecab.morphs(text2)
```

```
☞ ['학생', '이', '자연어', '처리', '수업', '을', '듣', '다']
```

```
okt.morphs(text1)
```

```
okt.morphs(text2)
```

```
okt.morphs(text2)
```

```
okt.morphs(text2)
```

```
kkma.morphs(text1)
```

```
kkma.morphs(text1)
```

```
Hnn.morphs(text1)
```

```
Hnn.morphs(text1)
```

```
text2 = 'A.I. 자연어 처리가 재미있음. 문제는 영어보다 한국어로 할 때 너무 어려워요. 해보면 알겠?'
```

```
import re
```

```
re.split('W.',text2)
```

```
re.split('W.',text2)
```

```
kkma.sentences(text2)
```

```
kkma.sentences(text2)
```

```
mecab.sentences(text2)
```

```
AttributeError                                Traceback (most recent call last)
<ipython-input-67-4832605742fb> in <cell line: 1>()
----> 1 mecab.sentences(text2)

AttributeError: 'Mecab' object has no attribute 'sentences'
```

```
sentence = '애들이 A. I. 수업 이해했다고 해줘. 그리고 나서 수업 끝내자.'
print(kkma.sentences(sentence))
```

```
kkma.sentences(sentence)
```

✓ 한국어 단어 토큰 대신 형태소 토큰

```
sentence = '그 교실에서 성호가 자연어처리 수업을 한다.'
```

```
print(kkma.morphs(sentence))# 동사-형용사 어간-어미
```

```
kkma.morphs(sentence)
```

```
print(oka.morphs(sentence))# 동사-형용사 어간-어미
```

```
oka.morphs(sentence)
```

```
print(Hnn.morphs(sentence))
```

```
Hnn.morphs(sentence)
```

```
sentence2 = 'IP: 192.168.51.31 서버로 접속 후 파일 변경'
```

```
print(mecab.morphs(sentence2))# 동사-형용사 어간-어미
```

```
['IP', ':', '192', '.', '168', '.', '51', '.', '31', '서버', '로', '접속', '후', '파일', '변경']
```

```
|print(kkma.morphs(sentence2))# 동상-형용사 어간-어미
```

```
['IP', ':', '192.168', '.', '51.31', '서버', '로', '접속', '후', '파일', '변경']
```

```
print(oka.morphs(sentence2))# 동상-형용사 어간-어미
```

```
['IP', ':', '192.168', '.', '51.31', '서버', '로', '접속', '후', '파일', '변경']
```

```
print(Hnn.morphs(sentence2))# 동상-형용사 어간-어미
```

```
['IP', ':', '192.', '168.', '51.', '31', '서버', '로', '접속', '후', '파', '이', 'ㄹ', '변경']
```

```
sentence3 = '애들아 A. I. 수업 이해했다고 해줘. 그리고 나서 수업 끝내자.'
```

```
print(kkma.morphs(sentence3))# 동상-형용사 어간-어미 '하, 었, 다고'
```

```
['애', '들', '아', 'A', '.', 'I', '.', '수업', '이해', '하', '었', '다고', '하', '어', '주', '어', '.', '그리고', '나', '서', '수
```

```
print(oka.morphs(sentence3))# 동상-형용사 어간-어미
```

```
['애', '들', '아', 'A', '.', 'I', '.', '수업', '이해', '했다고', '해줘', '.', '그리고', '나서', '수업', '끝내자', '.']
```

```
print(Hnn.morphs(sentence3))# '하, 었다'
```

```
['애들', '아', 'A', '.', 'I', '.', '수업', '이해', '하', '었다', '고', '하', '어', '주', '어', '.', '그리고', '나', '서', '수업',
```

코딩을 시작하거나 시로 코드를 생성하세요.

▼ pos tagging

```
print(kkma.pos(sentence3))
```

```
[('애', 'NNG'), ('들', 'XSN'), ('아', 'JKI'), ('A', 'OL'), ('.', 'SF'), ('I', 'OL'), ('.', 'SF'), ('수업', 'NNG'), ('이해', 'NNG')]
```

Image('./꼬꼬마 품사 태깅표.png')

대분류	새종 품사 태그		검정품 품사 태그		KKMA 단일 태그 V 1.0		확률태그	제공사전
	태그	설명	Class	설명	유형 1	유형 2	태그	설명
체언	NNG	일반 명사	NN	명사	N	NR	NNG	보통 명사
	NAP	고유 명사					NAP	고유 명사
	NVB	의문 명사	NX	의문 명사			NVB	일반 의문 명사
			UM	일체 명사			NNM	단위 의문 명사
	NR	수사	NU	수사			NR	수사
	NP	대명사	NP	대명사			NP	대명사
용언	VV	동사	VV	동사	V	VC	VV	동사
	VA	형용사	AJ	형용사			VA	형용사
	VX	보조 동사	VX	보조 동사			VXV	보조 동사
			AX	보조 형용사			VXA	보조 형용사
	VCP	명령 지명사	CP	서술격 조사			VCP	명령 지명사, 서술격 조사
	VCN	부정 지명사		의태			VCN	부정 지명사, 형용사 '아니'
관형사	MM	관형사	DT	일반 관형사	M	MD	MDT	일반 관형사
			DN	우 관형사			MDN	우 관형사
부사	MAG	일반 부사		부사	MA	MA	MAG	일반 부사
	MAJ	같은 부사	AD	부사			MAJ	같은 부사
감탄사	IC	감탄사	EX	감탄사	I	IC	IC	감탄사
조사	JKS	주격 조사	JO	조사	J	JK	JKS	주격 조사
	JKO	목적격 조사					JKO	목적격 조사
	JKQ	관형격 조사					JKQ	관형격 조사
	JKO	종격격 조사					JKO	종격격 조사
	JKB	부사격 조사					JKM	부사격 조사
	JKV	호격 조사					JKI	호격 조사
	JKQ	인용격 조사					JKQ	인용격 조사
	JX	보조사					JX	보조사
	JC	결합 조사					JC	결합 조사
							JC	결합 조사

Image('./꼬꼬마 품사 태깅표2.png')


```

print("-----")
print(Oket.nouns(sentence))
print(" ")
print("품사 태킹(PoS)")
print("-----")
print(Oket.pos(sentence))

print("형태소 단위로 문장 분리")
print("-----")
print(mecab .morphs(sentence))
print(" ")
print("문장에서 명사 추출")
print("-----")
print(mecab .nouns(sentence))
print(" ")
print("품사 태킹(PoS)")
print("-----")
print(mecab .pos(sentence))

```

↻ 형태소 단위로 문장 분리

['A', '.', 'I', '.', '자연어', '처리', '가', '재미있', '음', '.', '문제', '는', '영어', '보다', '한국어', '로', '할', '때', '너무']

문장에서 명사 추출

['자연어', '처리', '문제', '영어', '한국어', '때']

품사 태킹(PoS)

[('A', 'SL'), ('.', 'SY'), ('I', 'SL'), ('.', 'SY'), ('자연어', 'NNG'), ('처리', 'NNG'), ('가', 'JKS'), ('재미있', 'VA'), ('음',

코딩을 시작하거나 AI로 코드를 생성하세요.

✓ 정규표현식

```
import re
```

✓ (A) re.split() 분리함수

정규식과 매칭되는 문자열들을 분리하여 리스트로 리턴합니다.

```
text = "교수님이 자연어처리 수업을 한다"
```

```
print(re.split(" ",text))
```

↻ ['교수님이', '자연어처리', '수업을', '한다']

```
text = "교수님이, 자연어처리,수업을 한다"
```

```
#the backslash character ('\W') to indicate special forms
print(re.split("\W",text))
```

↻ ['교수님이', ' 자연어처리', '수업을 한다']

```
text = """교수님이
자연어처리
수어를
한다"""
```

```
print(re.split("\Wn",text))
```

↻ ['교수님이', '자연어처리', '수어를', '한다']

```
text="교수님이자연어처리+수업을한다"
re.split("W+",text)
```

```
['교수님이', '자연어처리', '수업을', '한다']
```

```
text="교수님이-자연어처리-수업을-한다"
re.split("W-",text)
```

```
['교수님이', '자연어처리', '수업을', '한다']
```

▼ (B) re.findall() 분리함수

문자열들을 분리하여 리스트로 리턴합니다.

```
text="""이름 : 박성호
전화번호 : 010 - 1234 - 1234
나이 : 30
성별 : 남"""
```

```
re.findall("Wd",text)
```

```
['0', '1', '0', '1', '2', '3', '4', '1', '2', '3', '4', '3', '0']
```

```
#[]안에 문자 찾기
re.findall("[Wd]",text)
```

```
['0', '1', '0', '1', '2', '3', '4', '1', '2', '3', '4', '3', '0']
```

```
re.findall("[0-9]",text)
```

```
['0', '1', '0', '1', '2', '3', '4', '1', '2', '3', '4', '3', '0']
```

```
#최소 1번이상 반복
re.findall("Wd+",text)
```

```
['010', '1234', '1234', '30']
```

```
re.findall("[0-9]+",text)
```

```
['010', '1234', '1234', '30']
```

```
#숫자가 아닌거과 매칭해서 리스트로
#re.findall("WD",text)
#re.findall("WD+",text)
```

```
#^ not 의미
#re.findall("[^0-9]+",text)
#re.findall("[^Wd]+",text)
```

```
#문자 숫자와 매칭
#re.findall("[Ww]+",text)
#re.findall("[가-힣0-9]+",text)
#re.findall("[a-zA-Z0-9]+",'Abdsakdj0909')
```

```
from IPython.display import Image
Image('./자주쓰이는 정규식 문자 클래스.png')
```

🔗 [자주 사용하는 문자 클래스]

[0-9] 또는 [a-zA-Z] 등은 무척 자주 사용하는 정규 표현식이다. 이렇게 자주 사용하는 정규식은 별도의 표기법으로 표현할 수 있다. 다음을 기억해 두자.

- `\d` - 숫자와 매치, [0-9]와 동일한 표현식이다.
- `\D` - 숫자가 아닌 것과 매치, `^[^0-9]` 와 동일한 표현식이다.
- `\s` - whitespace 문자와 매치, `[\t\n\r\f\v]` 와 동일한 표현식이다. 맨 앞의 빈 칸은 공백문자(space)를 의미한다.
- `\S` - whitespace 문자가 아닌 것과 매치, `^[^ \t\n\r\f\v]` 와 동일한 표현식이다.
- `\w` - 문자+숫자(alphanumeric)와 매치, `[a-zA-Z0-9_]` 와 동일한 표현식이다.
- `\W` - 문자+숫자(alphanumeric)가 아닌 문자와 매치, `^[^a-zA-Z0-9_]` 와 동일한 표현식이다.

대문자로 사용된 것은 소문자의 반대임을 추측할 수 있다.

```
text = "나는는 최고이다 "
```

```
re.findall("[^가-힣]+$", text)
```

```
🔗 [' ']
```

```
Image('많이 쓰이는 정규식.png')
```

🔗 정규 표현식

- Email 주소
`^\w+([-+.] \w+)*@\w+([-+.] \w+)*\.\w+([-+.] \w+)*$`
- 도메인 이름
`^[a-zA-Z0-9][a-zA-Z0-9-]{1,61}[a-zA-Z0-9]\.[a-zA-Z]{2,}$`
- 인터넷 URL
`^(http|https)://([ \w-]+\.)+[ \w-]+(/[ \w-./?%&=]*)?$`
- 사용자 이름 (Twitter username)
`^[a-zA-Z0-9_ \-]{3,15}$`
- 유선 전화
`^0\d{4}-\d{4}`
- 휴대 전화
`^(010|019|011)-\d{4}-\d{4}`
- 인터넷 전화
`^070-\d{4}-\d{4}`

```
#(C) re.sub() 표현식 패턴과 일치하는 문자열을 찾아 다른 문자열로 대체
text="ADAS[1]"
```

```
re.sub('[^a-zA-Z]', '', text)
```

```
↔ 'ADAS'
```

```
text="이 영화 개웃김 ㅋㅋㅋ!!!"
re.sub('[WWW]', '', text)
```

```
↔ '이영화개웃김ㅋㅋㅋ'
```

```
re.sub('[^가-힣0-9]+', '', text)
```

```
↔ '이영화개웃김'
```

```
re.sub('[^가-힣0-9]+$', '', text)
```

```
↔ '이 영화 개웃김'
```

✓ Stop words 불용어

```
text = "고기를 아무렇게나 구우려고 하면 안돼. 고기라고 다 같은 게 아니거든"
```

```
from konlpy.tag import Kkma
```

```
km = Kkma()
```

```
korean_mophs = km.morphs(text)
```

```
stop_words = "때"
```

```
token_set = []
for token, pos in results:
    if pos in ['NNG', 'VA']:
        if token not in stop_words:
            token_set.append(token)
```

```
token_set
```

```
↔ ['자연어', '처리', '재미있', '문제', '영어', '한국어', '어렵']
```

```
result = []
for w in korean_mophs:
    if w not in stop_words:
        result.append(w)
```

```
print(korean_mophs)
```

```
↔ ['고기', '를', '아무렇게나', '게', '나', '아', '굽', '려고', '하', '면', '안', '되', '어', '.', '고기', '라고', '다', '같', '은', '것']
```

```
print(result)
```

```
↔ ['고기', '를', '굽', '려고', '안', '되', '어', '.', '고기', '라고', '같', '은', '것', '이', '아니', '거든']
```

```
import os
```

```
os.getcwd()
```

```
↔ '/content'
```

```
stopwords = []
with open('./한국어불용어100.txt', 'r', encoding='utf-8') as f:
    for line in f:
        stopwords.append(line.rstrip('\n'))
```



```
-----
FileNotFoundError                                Traceback (most recent call last)
<ipython-input-47-d4c591024d8c> in <cell line: 2>()
      1 stopwords = []
----> 2 with open('./한국어불용어100.txt', 'r', encoding='utf-8') as f:
      3     for line in f:
      4         stopwords.append(line.rstrip('\n'))
```

```
FileNotFoundError: [Errno 2] No such file or directory: './한국어불용어100.txt'
```

stopwords



```
['이',
 '있',
 '하',
 '것',
 '들',
 '그',
 '되',
 '수',
 '이',
 '보',
 '않',
 '없',
 '나',
 '사람',
 '주',
 '아니',
 '등',
 '같',
 '우리',
 '때',
 '년',
 '가',
 '한',
 '지',
 '대하',
 '오',
 '말',
 '일',
 '그렇',
 '위하',
 '때문',
 '그것',
 '두',
 '말하',
 '알',
 '그러나',
 '받',
 '못하',
 '일',
 '그런',
 '또',
 '문제',
 '더',
 '사회',
 '많',
 '그리고',
 '줄',
 '크',
 '따르',
 '중',
 '나오',
 '가지',
 '씨',
 '시키',
 '만들',
 '지금',
 '생각하',
 '그러',
```

```
result = []
for w in korean_mophs:
    if w not in stopwords:
        result.append(w)
```

```
print(korean_mophs)
```

```
['고기', '를', '아무렇게', '게', '나', '아', '굽', '려고', '하', '면', '안', '되', '어', '.', '고기', '라고', '다', '같', '은', '것']
```

```
print(result)
```

```
['고기', '를', '아무렇게', '게', '아', '굽', '려고', '면', '어', '.', '고기', '라고', '다', '은', '거든']
```

```
import re
```

```
'''
```

1. 영어 소문자와 한글을 제외한 모든 문자를 제거
 2. Okt를 이용해 형태소 분석
 3. 형태소 분석기를 거쳐 나온 결과들 중 stopwords 리스트에 포함되지 않는 토큰만 token이라는 리스트에 반환
- ```
'''
```

```
tokenizer = mecab
```

```
def text_preprocessing(text, tokenizer):
```

```
 stopwords = ['을', '를', '이', '가', '은', '는']
```

```
 txt = re.sub('[^가-힣a-z]', ' ', text)
```

```
 token = tokenizer.morphs(txt)
```

```
 token = [t for t in token if t not in stopwords]
```

```
 return token
```

```
example_pre = text_preprocessing(sentence, tokenizer)
```

```
example_pre
```

```
['자연어',
 '처리',
 '재미있',
 '음',
 '문제',
 '영어',
 '보다',
 '한국어',
 '로',
 '할',
 '때',
 '너무',
 '어려워요',
 '해',
 '보',
 '면',
 '알',
 '결']
```

```
def text_preprocessing(text_list):
```

```
 stopwords = ['을', '를', '이', '가', '은', '는', 'null'] #불용어 설정
```

```
 tokenizer = Okt() #형태소 분석기
```

```
 token_list = []
```

```
 for text in text_list:
```

```
 txt = re.sub('[^가-힣a-z]', ' ', text) #한글과 영어 소문자만 남기고 다른 글자 모두 제거
```

```
 token = tokenizer.morphs(txt) #형태소 분석
```

```
 token = [t for t in token if t not in stopwords or type(t) != float] #형태소 분석 결과 중 stopwords에 해당하지 않는 것만 추출
 token_list.append(token)
```

```
 return token_list, tokenizer
```

## ✓ 유의어/동의어 표준화 시키기

비슷한 단어들을 표준화 하고 싶다면? 동의어 사전(단어사전)을 만들어 매핑시켜 표준화 하면 된다

# 예시 문장

```
sentence = "OMG 공부하기 넘 싫네요. 그래도 스터디 해야됩니다 오마이갓! 지자스 하 아~"
```

```
word_tokens = mecab.morphs(sentence)
```

```
rst_list = mecab.morphs(sentence)
```

```
import pandas as pd
```

```
synonym_dict = {
 '나' : ['저', '내'],
 '오마이갓' : ['OMG', 'Oh My God'],
 '공부' : ['스터디']
}
```

```
apply_mapping = {word:k for k, v in synonym_dict.items() for word in v}
result_list = pd.DataFrame(rst_list).replace(apply_mapping, regex=True).values
```

```
print('유의어 처리 전 : ', list(word_tokens))
print('유의어 처리 후: ', list(result_list.flatten()))
```

↔ 유의어 처리 전 : ['OMG', '공부', '하', '기', '넘', '싫', '네요', '.', '그래도', '스터디', '해야', '됩니다', '오마이갓', '!', '지'  
유의어 처리 후: ['오마이갓', '공부', '하', '기', '넘', '싫', '네요', '.', '그래도', '공부', '해야', '됩니다', '오마이갓', '!', '지'

코딩을 시작하거나 시로 코드를 선택하세요.